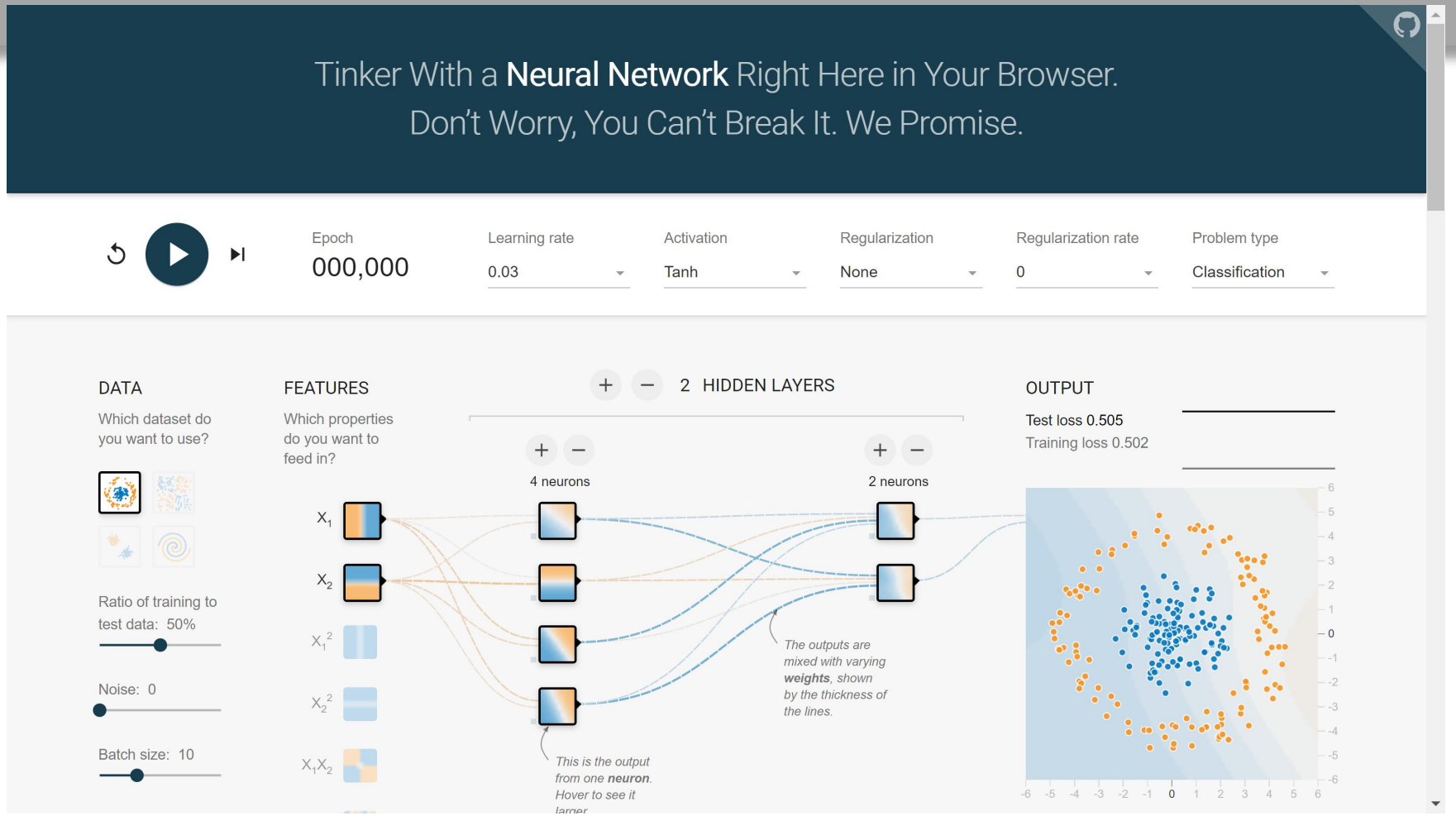
CS5187: Computer Vision

Neural networks



New problem: Image classification

- Input: an image
  - Assume it is of fixed size, e.g. 128x128 pixels
- Output: the class label for that image
- Label is generally a discrete label from a pre-defined set
  - e.g. {cat, dog, cow, toaster, apple, tomato, truck, … }

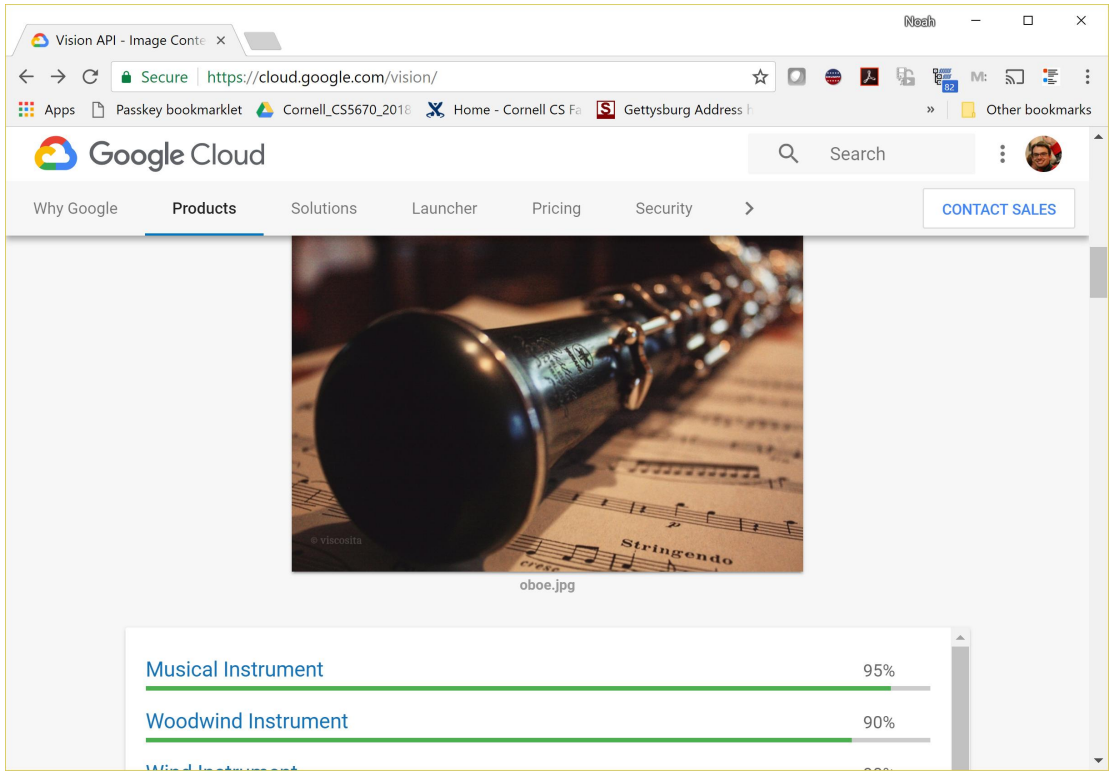
```
def classifier(image):
    //Do some stuff
    return class_label;
```

$f($  $) =$ "Cat"

$f($  $) =$ "Dog"

$f($  $) =$ "Toaster"

Image classification demo



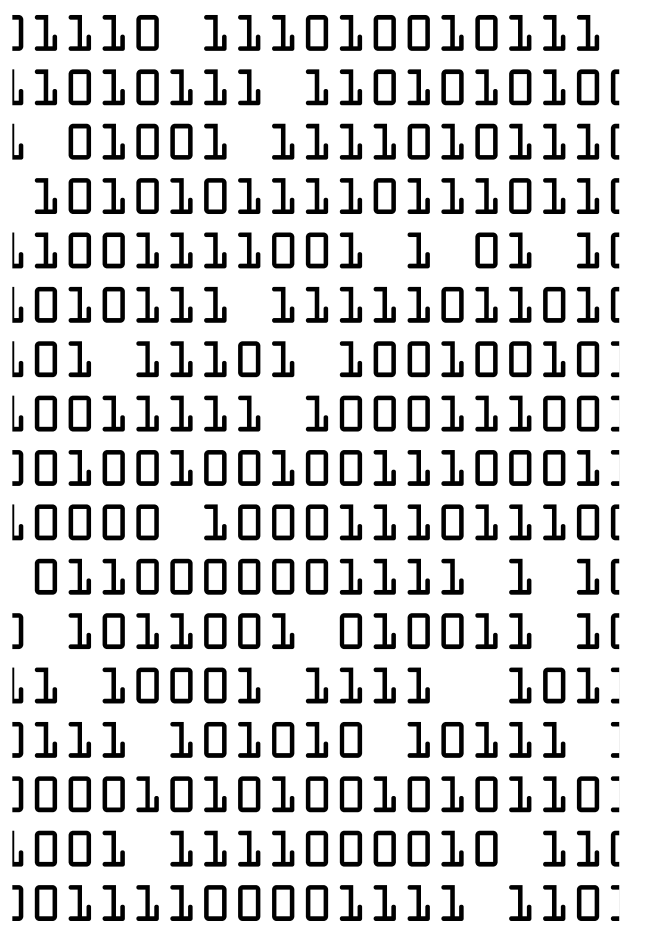
https://cloud.google.com/vision/docs/drag-and-drop

See also:
https://aws.amazon.com/rekognition/
https://www.clarifai.com/
https://azure.microsoft.com/en-us/services/cognitive-services/computer-vision/
…

The Semantic Gap



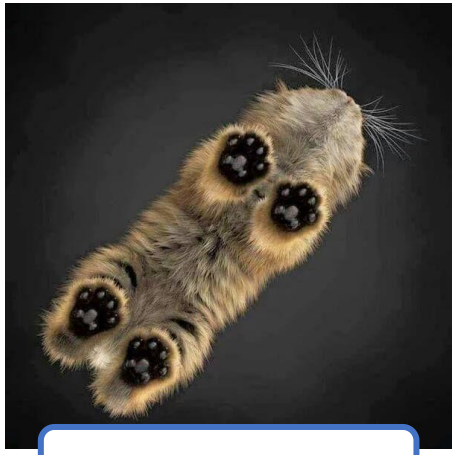
What we see



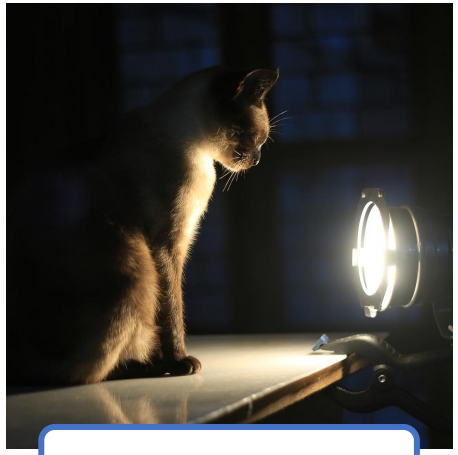
What the computer sees

Variation Makes Recognition Hard

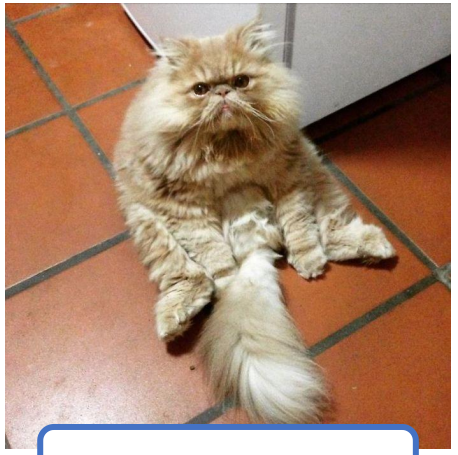
- The same class of object can appear *very* differently in different images



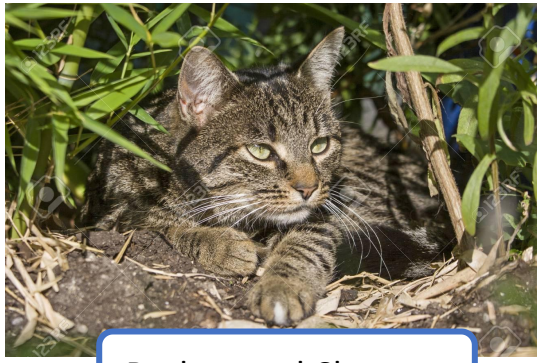
Viewpoint Variation



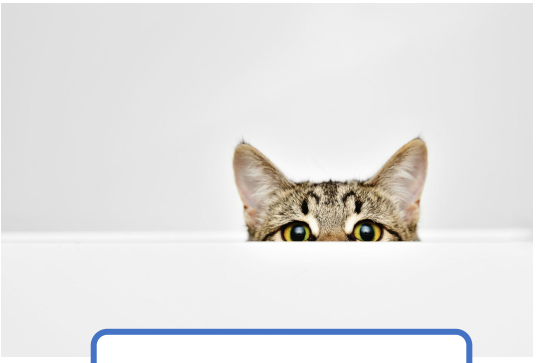
Lighting Variation



Deformation



Background Clutter



Occlusion

The Problem is Under-constrained

- Distinct realities can produce the same image…
- We generally can't compute the "right" answer, but we can compute the most likely one…
- We need some kind of prior to condition on. We can learn this prior from data:

$$f(x) = \underset{\ell_x}{argmax}\ P(\ell_x | data)$$





Images As High-Dimensional Vectors

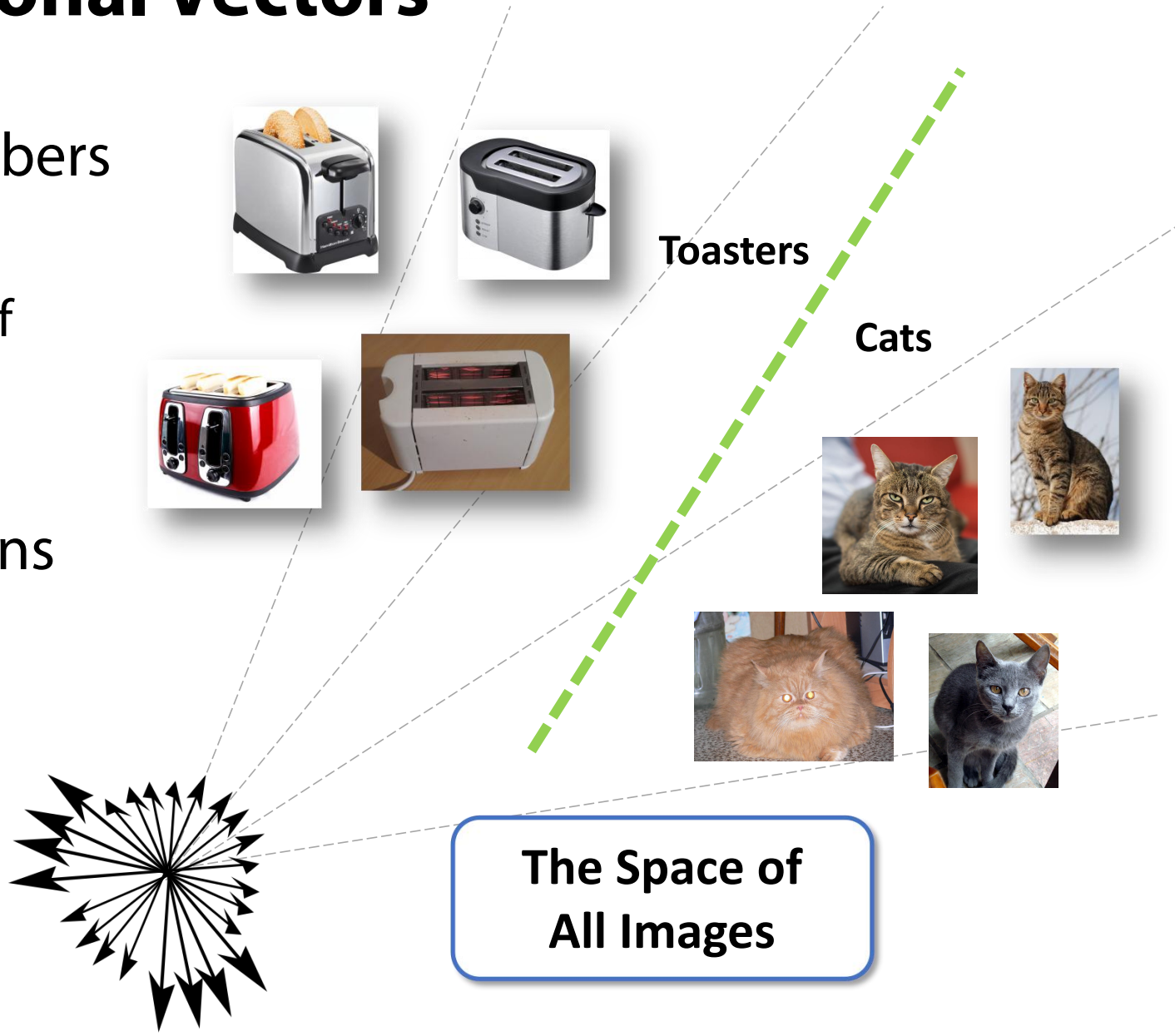
- An image is just a bunch of numbers
- Let's stack them up into a vector
  - Our training data is just a bunch of high-dimensional points now



Images As High-Dimensional Vectors

- An image is just a bunch of numbers
- Let's stack them up into a vector
  - Our training data is just a bunch of high-dimensional points now

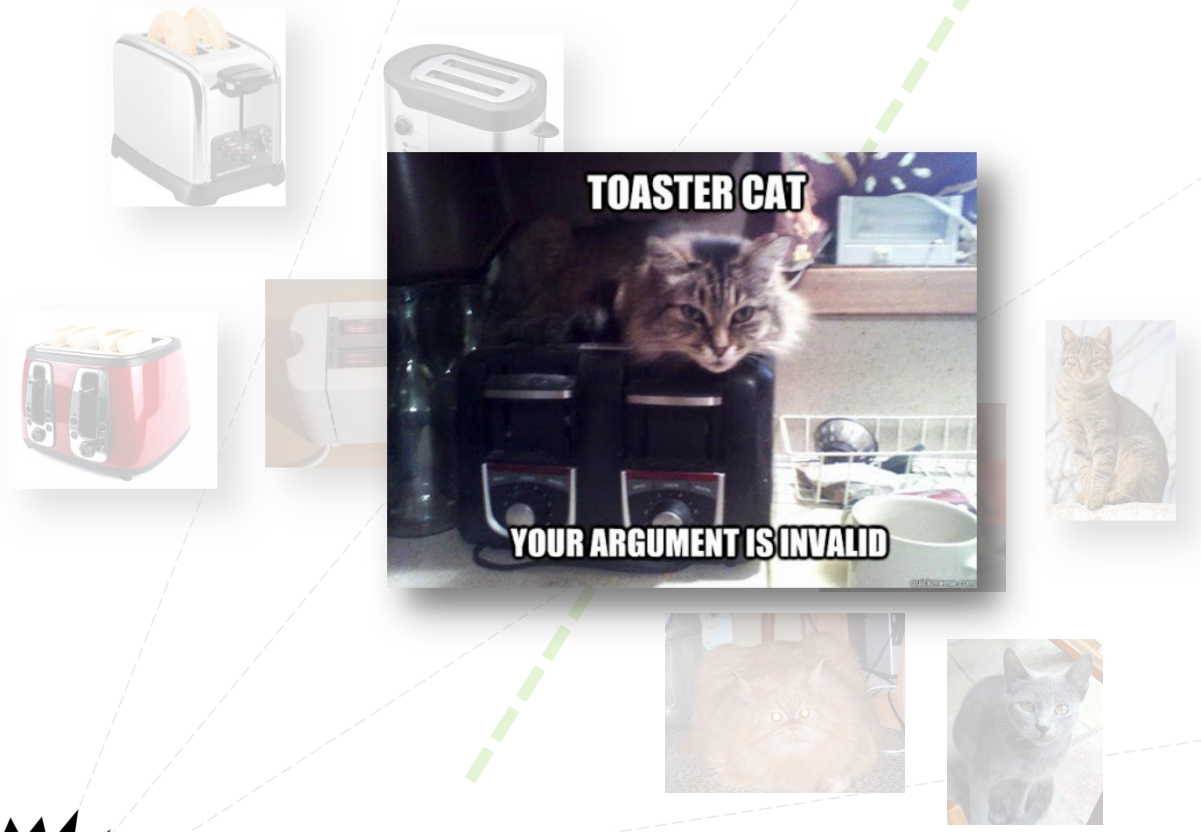
- Divide space into different regions for different classes

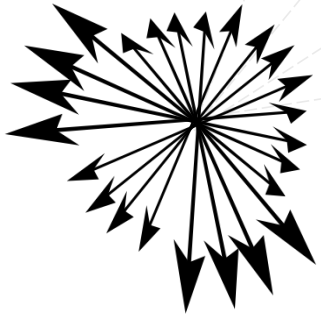


Images As High-Dimensional Vectors

- An image is just a bunch of numbers
- Let's stack them up into a vector
  - Our training data is just a bunch of high-dimensional points now

- Divide space into different regions for different classes





The Space of All Images

Images As High-Dimensional Vectors

- An image is just a bunch of numbers
- Let's stack them up into a vector
  - Our training data is just a bunch of high-dimensional points now

- Divide space into different regions for different classes

or

- Define a distribution over space for each class

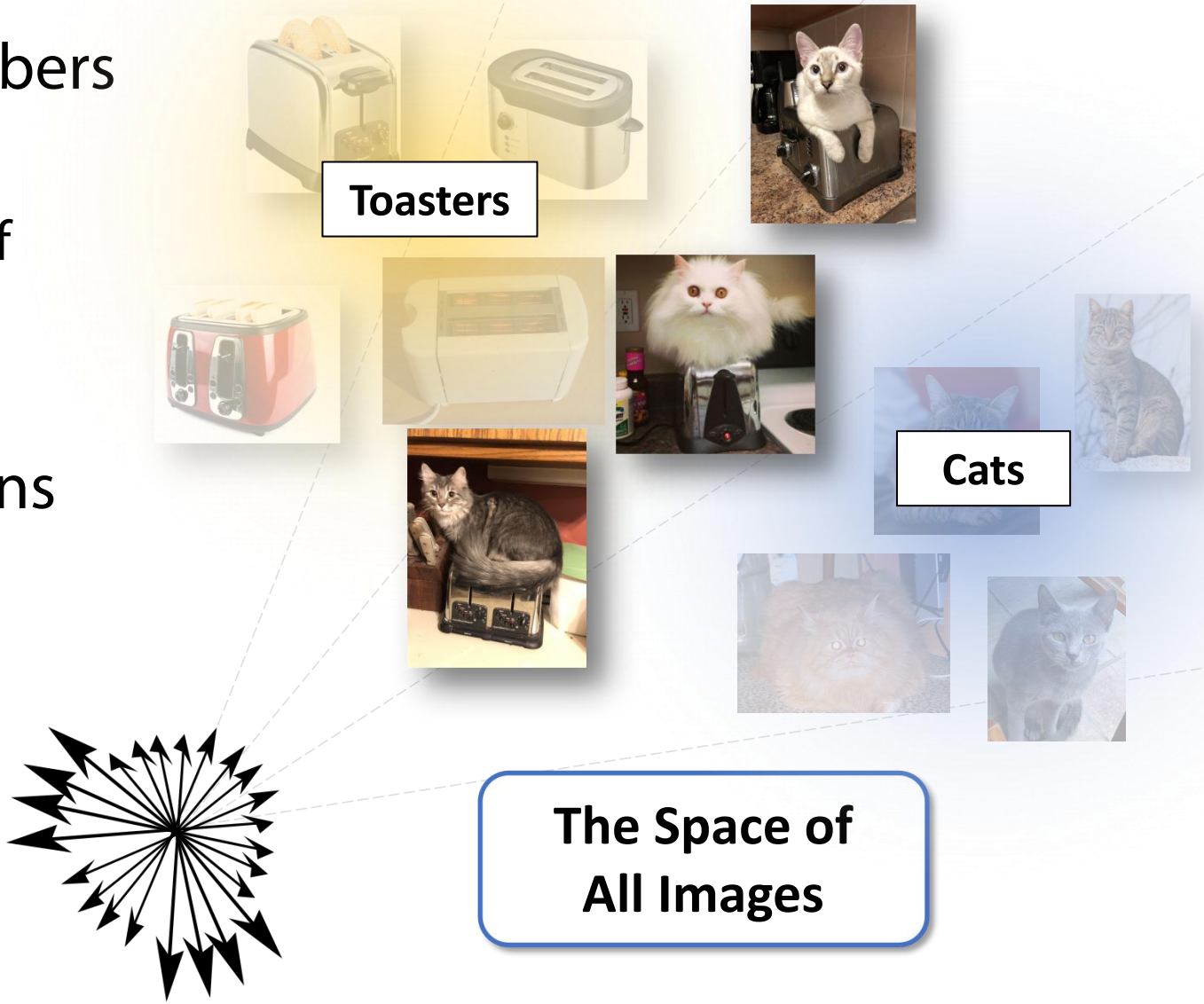


Image Features and Dimensionality Reduction

- How high-dimensional is an image?
  - Let's consider an iPhone X photo:
    - 4032 x 3024 pixels
    - Every pixel has 3 colors
    - 36,578,304 pixels (36.5 Mega pixels)
- In practice, images sit on a lower-dimensional manifold
- Think of image features and dimensionality reduction as ways to represent images by their location on such manifolds

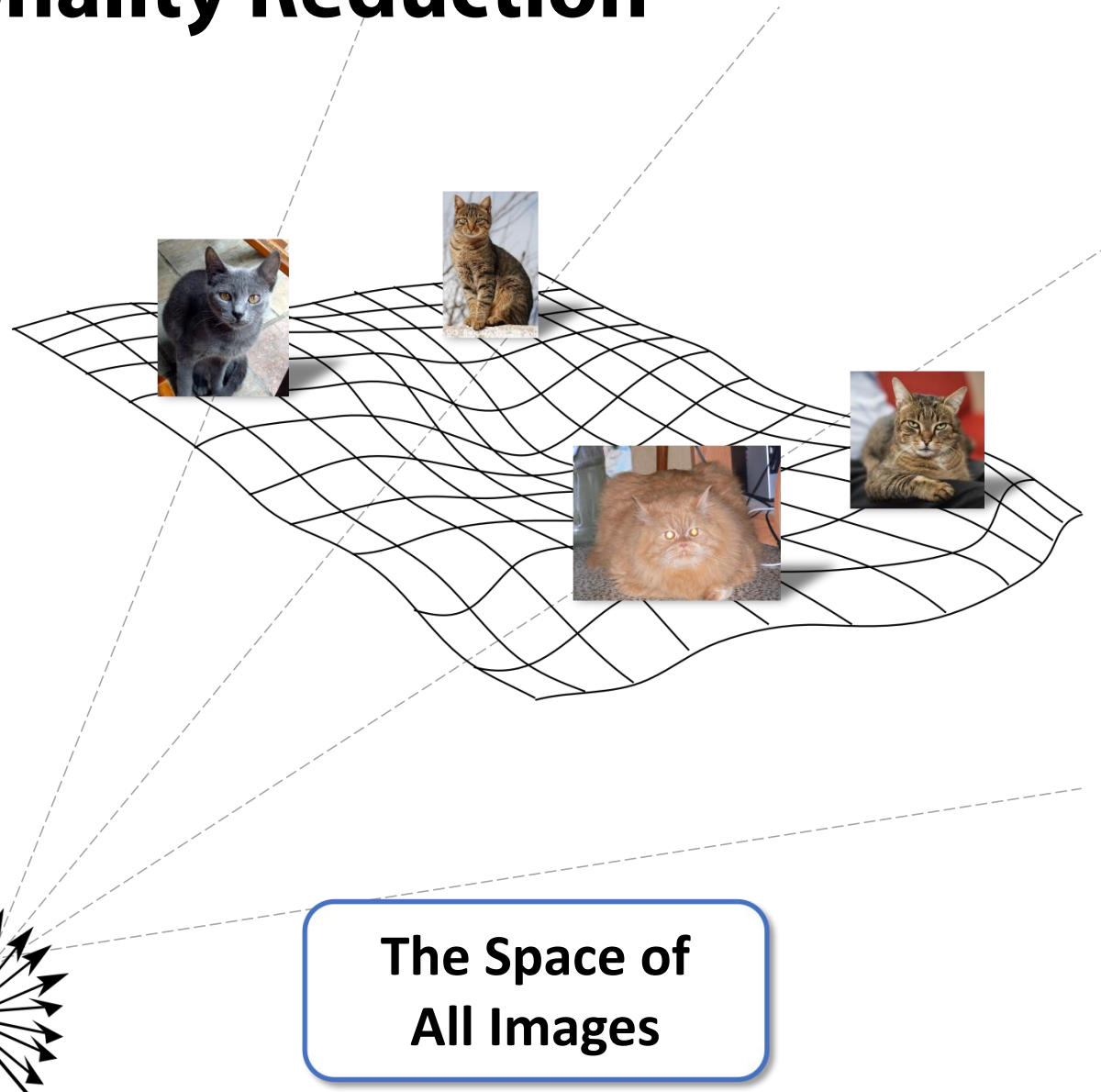
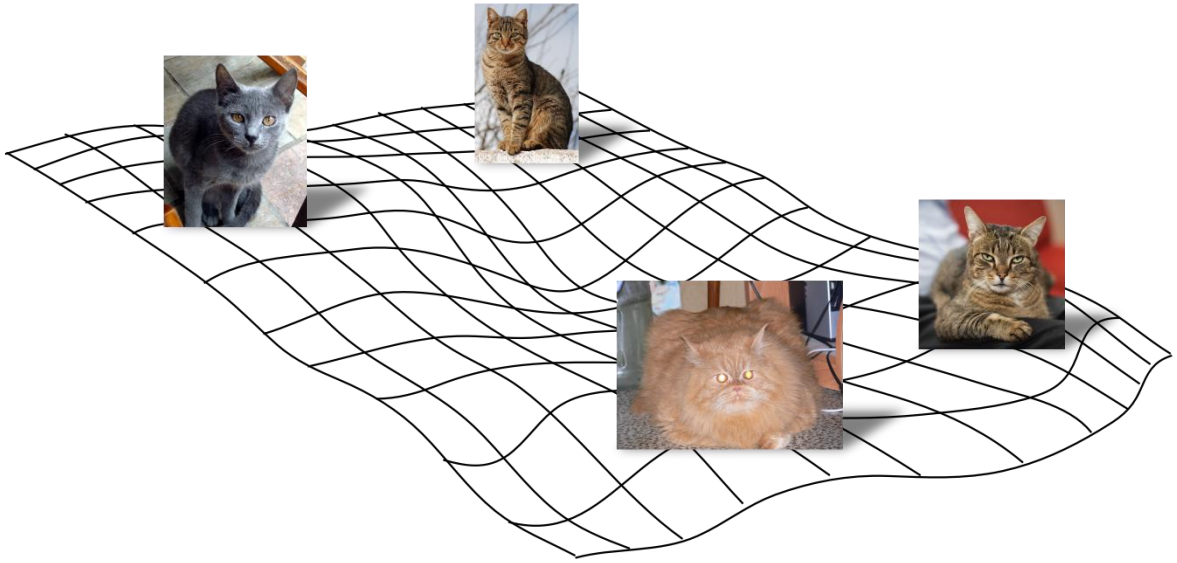


Image Features and Dimensionality Reduction

- How high-dimensional is an image?
  - Let's consider an iPhone X photo:
    - 4032 x 3024 pixels
    - Every pixel has 3 colors
    - 36,578,304 pixels (36.5 Mega pixels)
- In practice, images sit on a lower-dimensional manifold
- Think of image features and dimensionality reduction as ways to represent images by their location on such manifolds



Side Note:
This also lets us deal with images of different sizes, crops, etc.

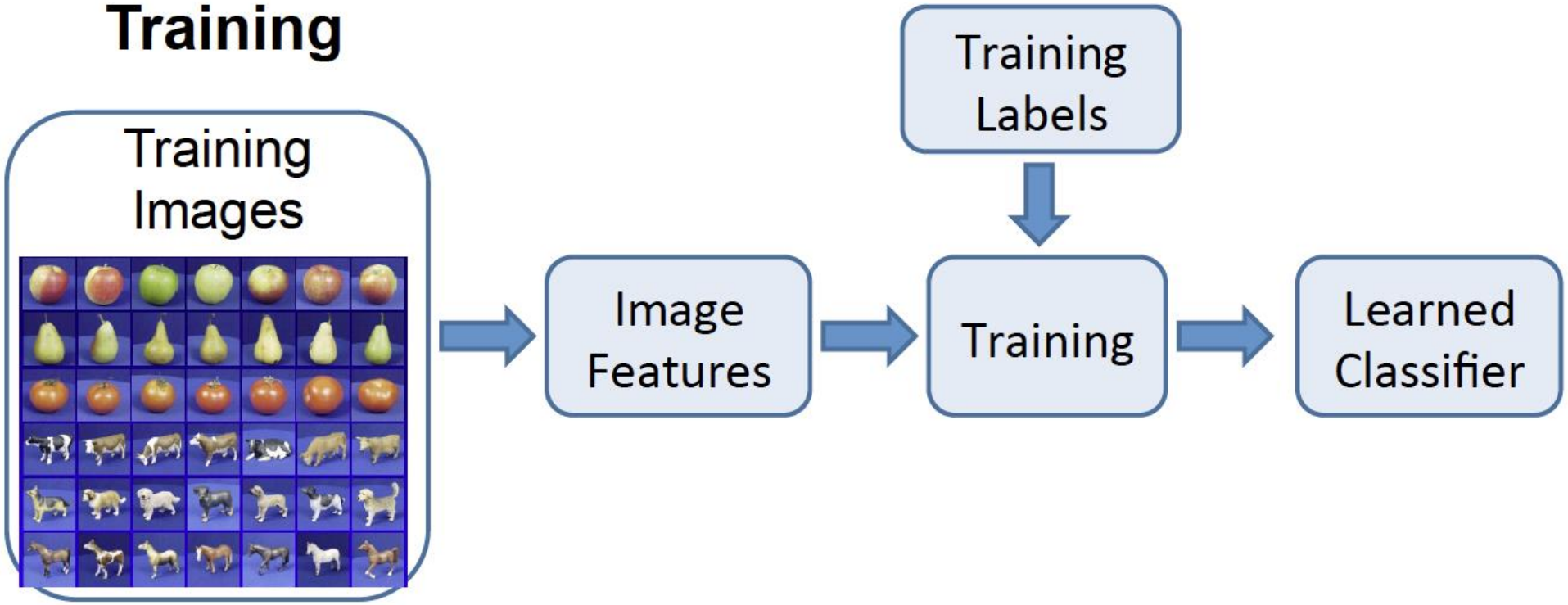
Training & Testing a Classifier

- Collect a database of images with labels
- Use ML to train an image classifier
- Evaluate the classifier on test images



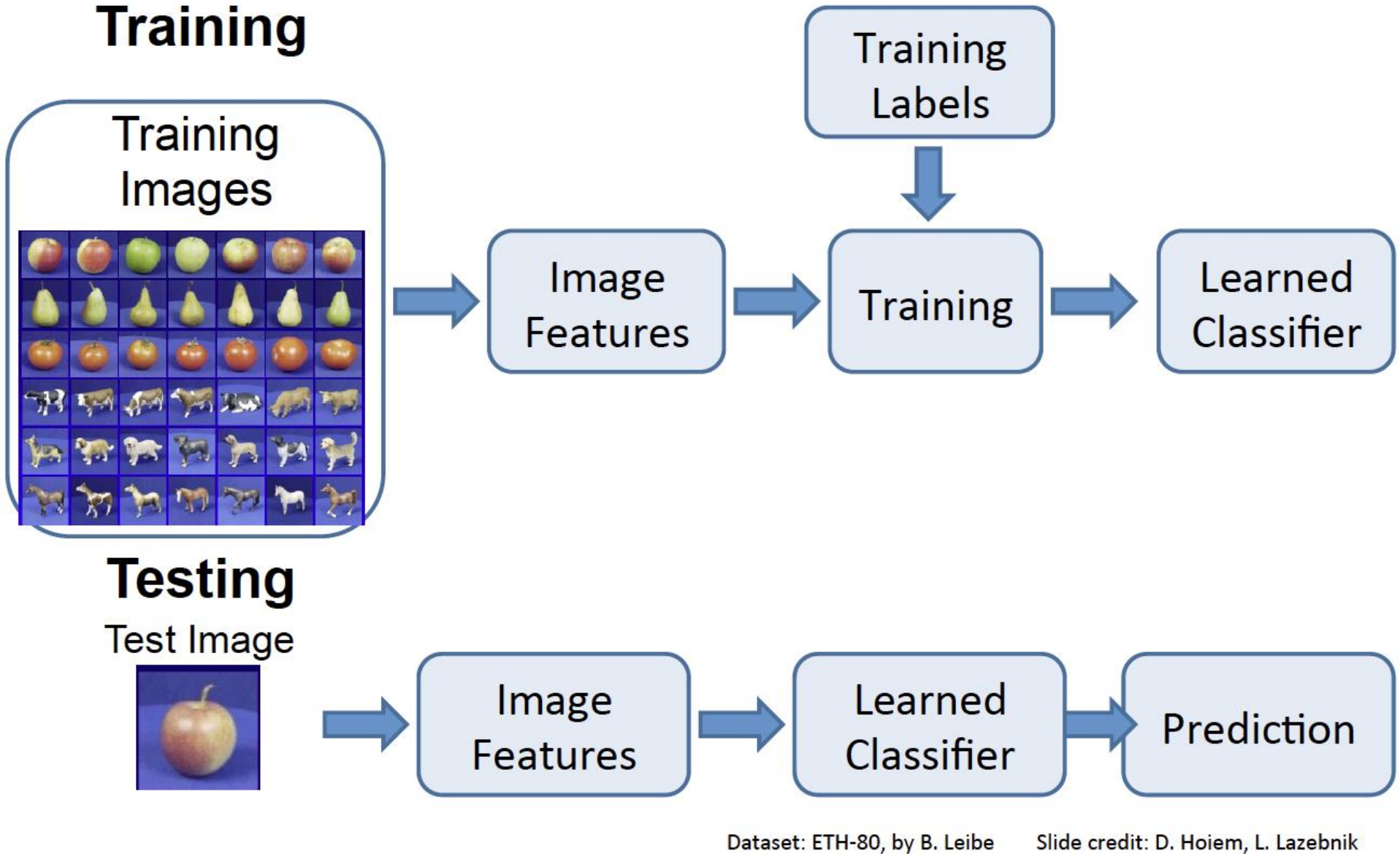
Slide from Andrej Karpathy and Fei-Fei Li http://vision.stanford.edu/teaching/cs231n/

Training & Testing a Classifier



Dataset: ETH-80, by B. Leibe Slide credit: D. Hoiem, L. Lazebnik

Training & Testing a Classifier



Dataset: ETH-80, by B. Leibe Slide credit: D. Hoiem, L. Lazebnik

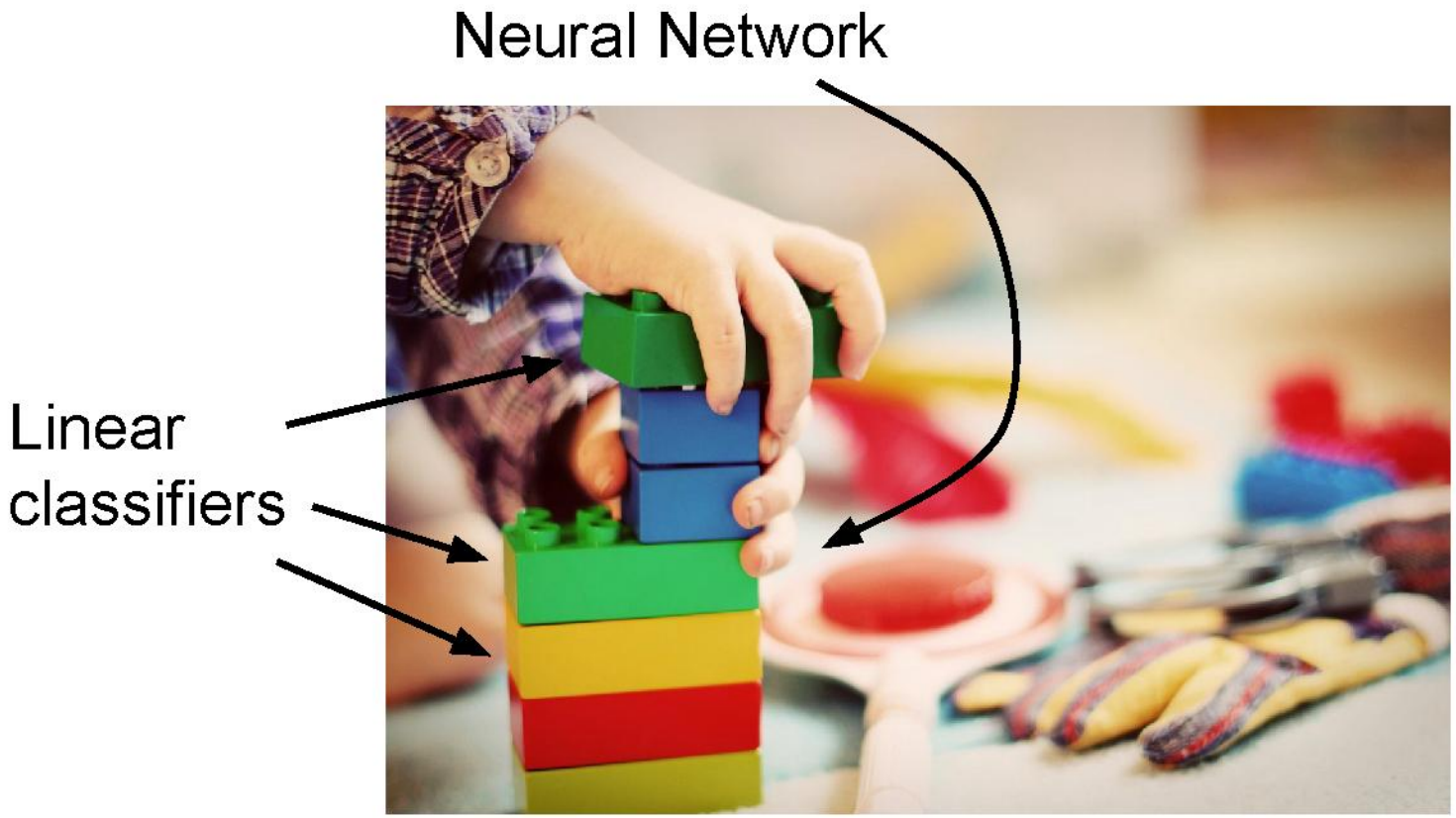
Classifiers

- Nearest Neighbor
- kNN (“k-Nearest Neighbors”)
- Linear Classifier
- Neural Network
- Deep Neural Network
- Transformers
- …

Classifiers

- Nearest Neighbor
- kNN (“k-Nearest Neighbors”)
- **Linear Classifier**
- **Neural Network**
- Deep Neural Network
- Transformers
- …

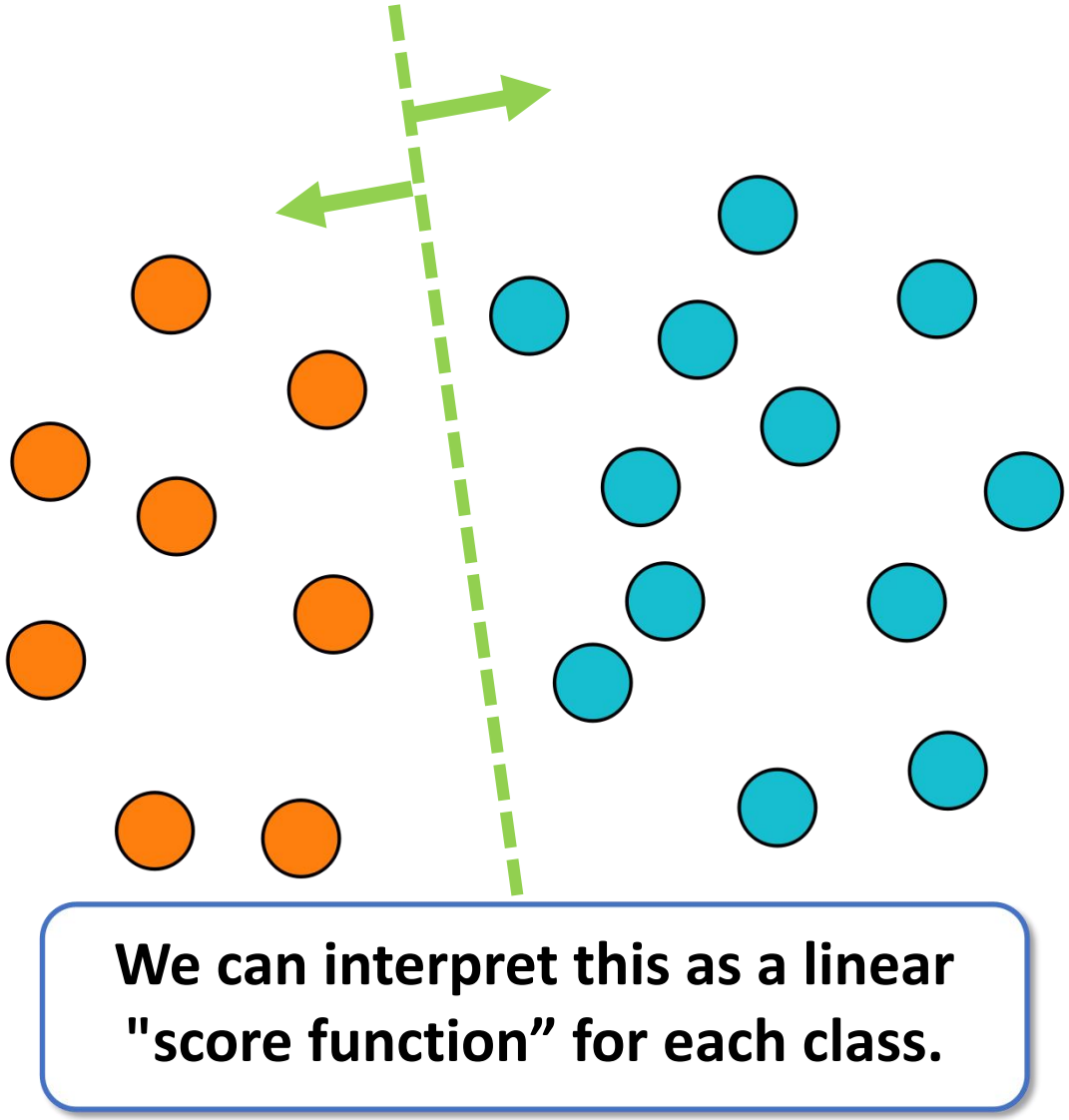
Linear Classifiers



This image is CC0 1.0 public domain

Linear Classification vs. Nearest Neighbors

- Linear Classifier
  - Store hyperplanes that best separate different classes
  - We can compute continuous class score by calculating (signed) distance from hyperplane



Score functions





class scores

Parametric Approach



image parameters

f(**x**,**W**)

10 numbers, indicating class scores

[32x32x3] $= 3072$
array of numbers 0...1
(3072 numbers total)

Parametric Approach: Linear Classifier

$$f(x, W) = Wx$$

3072x1

10x1

10x3072



10 numbers, indicating class scores

[32x32x3] $= 3072$
array of numbers 0...1

parameters, or “weights”

Slide adapted from Andrej Karpathy and Fei-Fei Li http://vision.stanford.edu/teaching/cs231n/

Parametric Approach: Linear Classifier

$f(x, W) = Wx$ + (+b)

3072x1

10x1

10x3072

10x1



10 numbers, indicating class scores

[32x32x3] $= 3072$
array of numbers 0...1

parameters, or “weights”

Slide adapted from Andrej Karpathy and Fei-Fei Li http://vision.stanford.edu/teaching/cs231n/

Linear Classifier

define a **score function**

data (image)

$$f(x_i, W, b) = Wx_i + b$$

class scores

"weights"

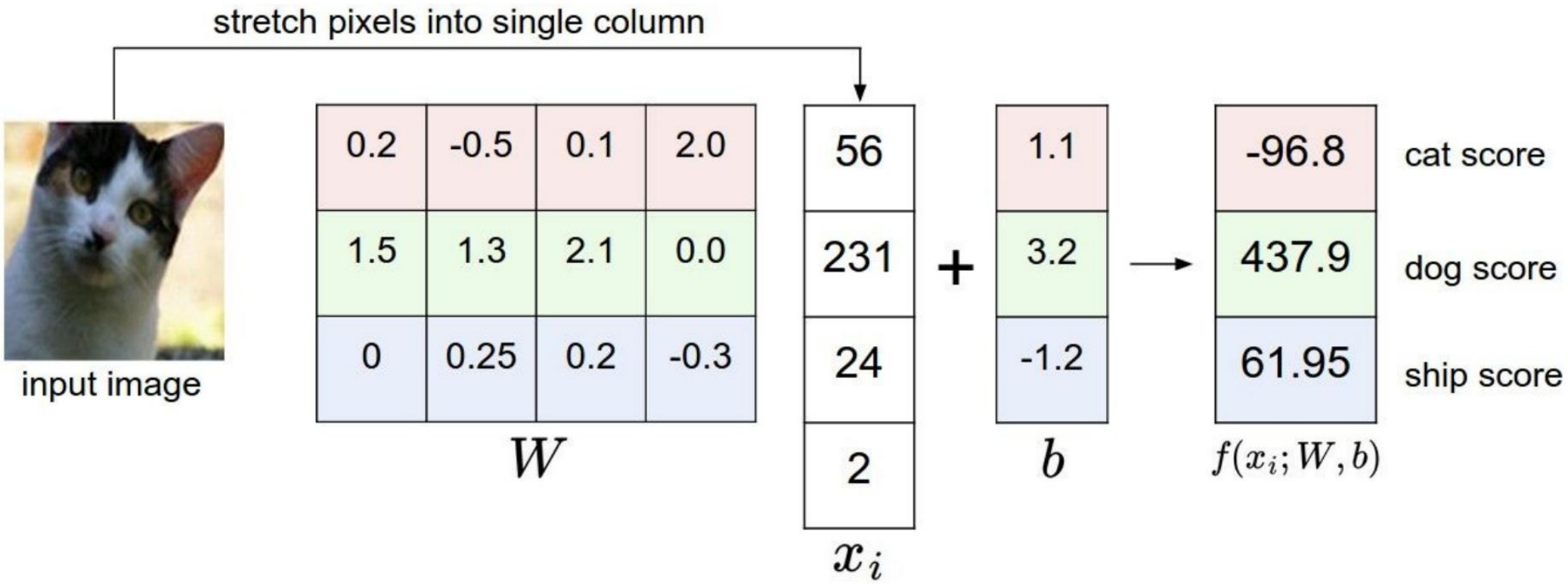
"bias vector"

"parameters"

Slide adapted from Andrej Karpathy and Fei-Fei Li http://vision.stanford.edu/teaching/cs231n/

Interpretation: Algebraic

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



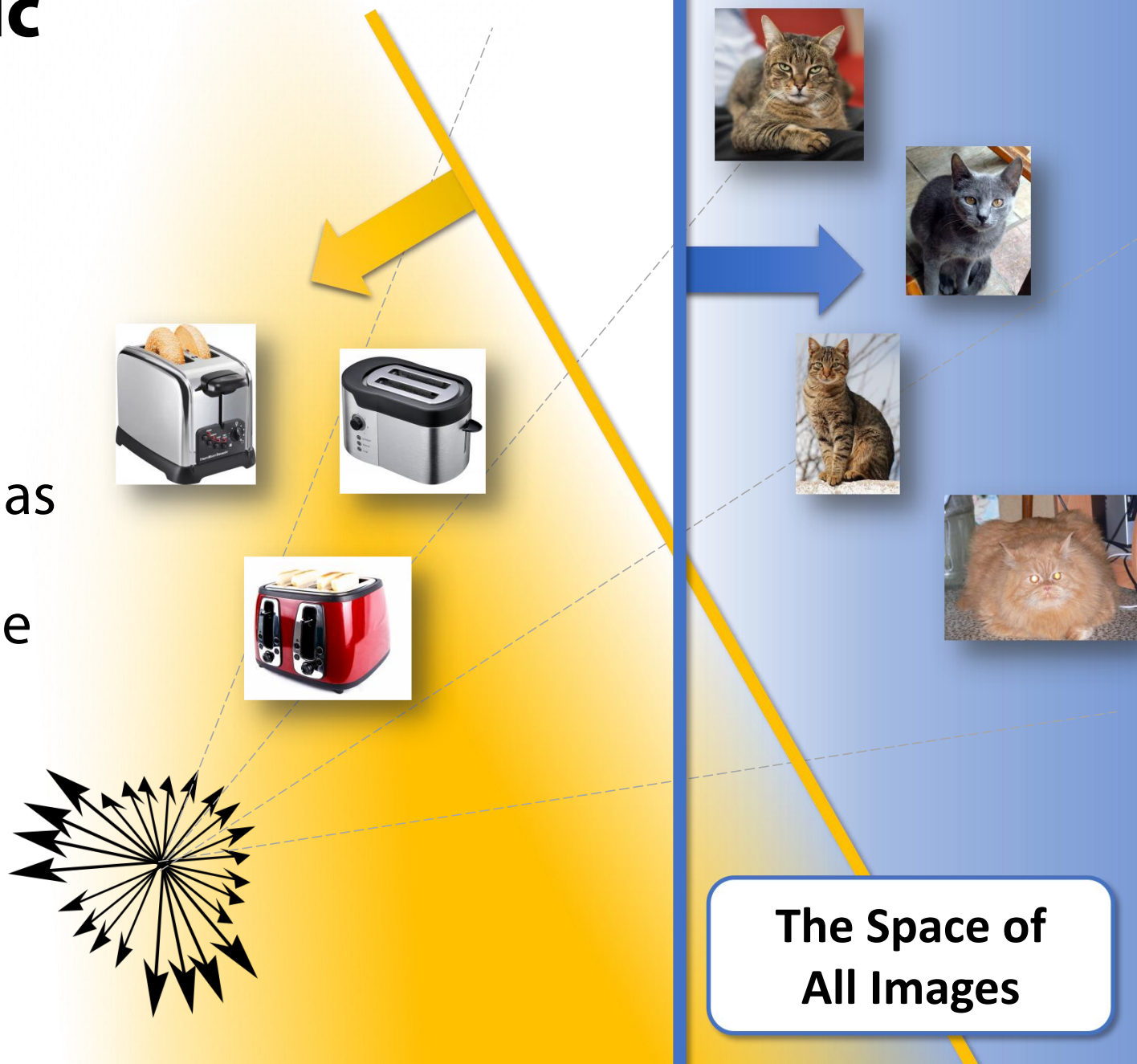
Slide adapted from Andrej Karpathy and Fei-Fei Li http://vision.stanford.edu/teaching/cs231n/

Interpretation: Geometric

- Parameters define a hyperplane for each class:

$$f(x_i, W, b) = Wx_i + b$$

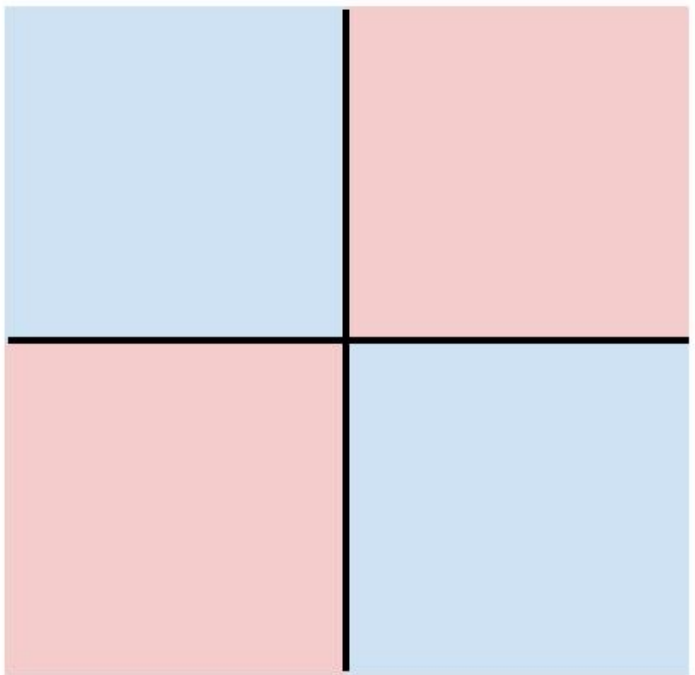
- We can think of each class score as defining a distribution that is proportional to distance from the corresponding hyperplane



Hard Cases for a Linear Classifier

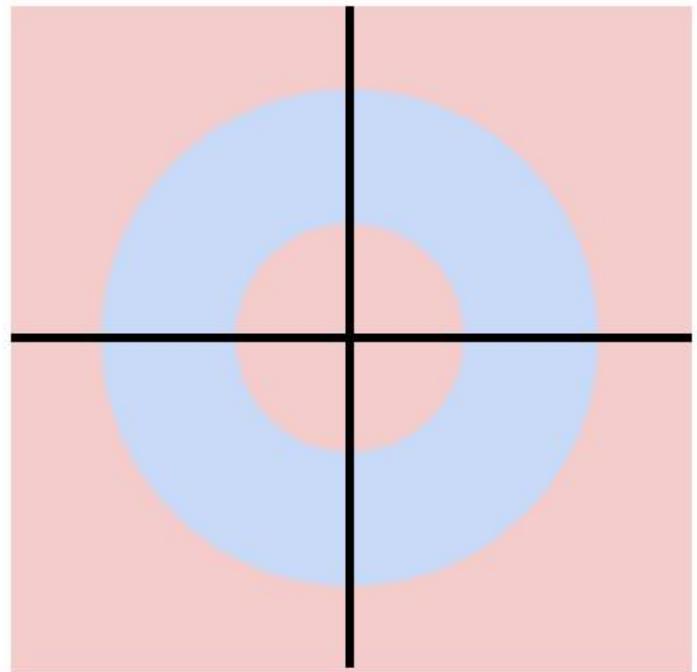
Class 1:
First and third quadrants

Class 2:
Second and fourth quadrants



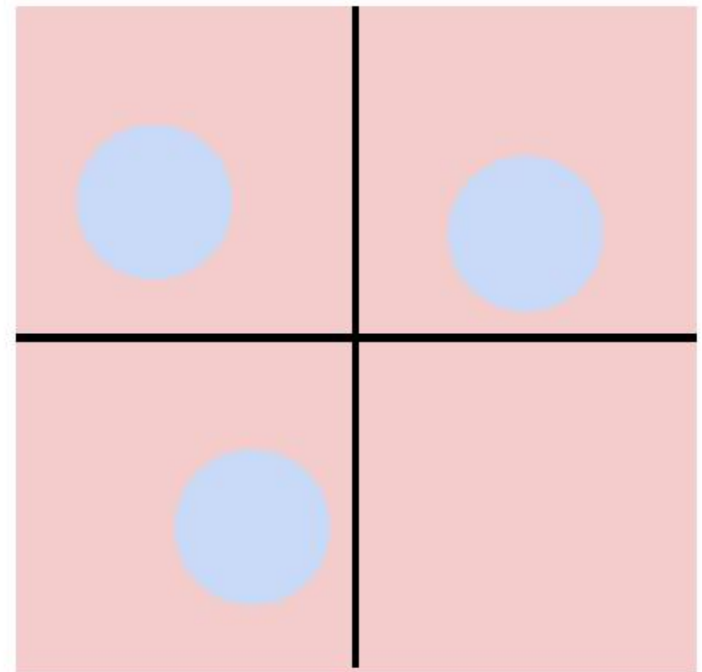
Class 1:
1 <= L2 norm <= 2

Class 2:
Everything else



Class 1:
Three modes

Class 2:
Everything else



Interpretation: Template matching

- We can think of the rows in W as templates for each class

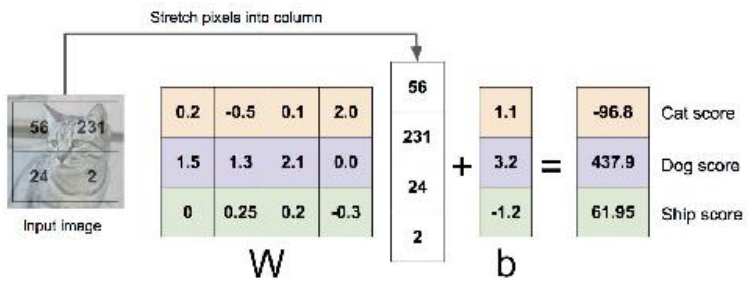


Rows of *W* in $f(x_i, W, b) = Wx_i + b$

Linear Classifier: Three Viewpoints

Algebraic Viewpoint

$$f(x,W) = Wx$$



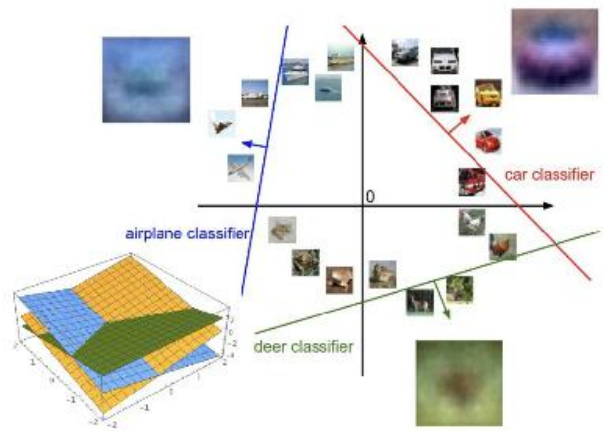
Visual Viewpoint

One template per class



Geometric Viewpoint

Hyperplanes cutting up space



So far: Defined a (linear) *score function* f(x, W) = Wx + b

Example class scores for 3 images for some W:

How can we tell whether this W is good or bad?





airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Cat image by Nikita is licensed under CC-BY 2.0
Car image is CC0 1.0 public domain
Frog image is in the public domain

Linear classification





airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

Output scores

TODO:

1. Define a **loss function** that quantifies our unhappiness with the scores across the training data.

2. Come up with a way of efficiently finding the parameters that minimize the loss function. **(optimization)**

Loss functions

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss over the dataset is a sum of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Loss function, cost/objective function

- Given ground truth labels (y_i), scores $f(x_i, \mathbf{W})$
  - how unhappy are we with the scores?
- Loss function or objective/cost function measures unhappiness

- During training, **want to find the parameters W that minimize the loss function**

Simpler example: binary classification

- Two classes (e.g., “cat” and “not cat”)
  - AKA “positive” and “negative” classes





cat

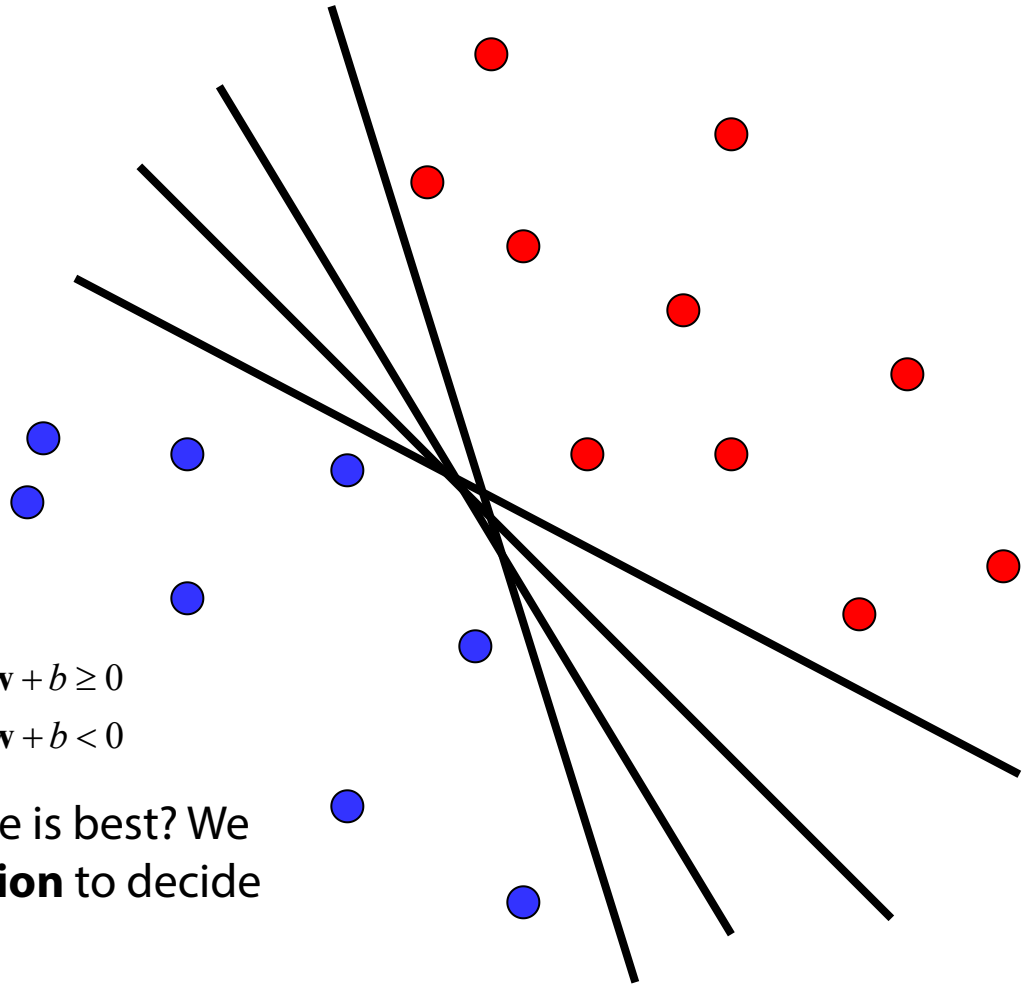




not cat

Linear classifiers

- Find linear function (*hyperplane*) to separate positive and negative examples

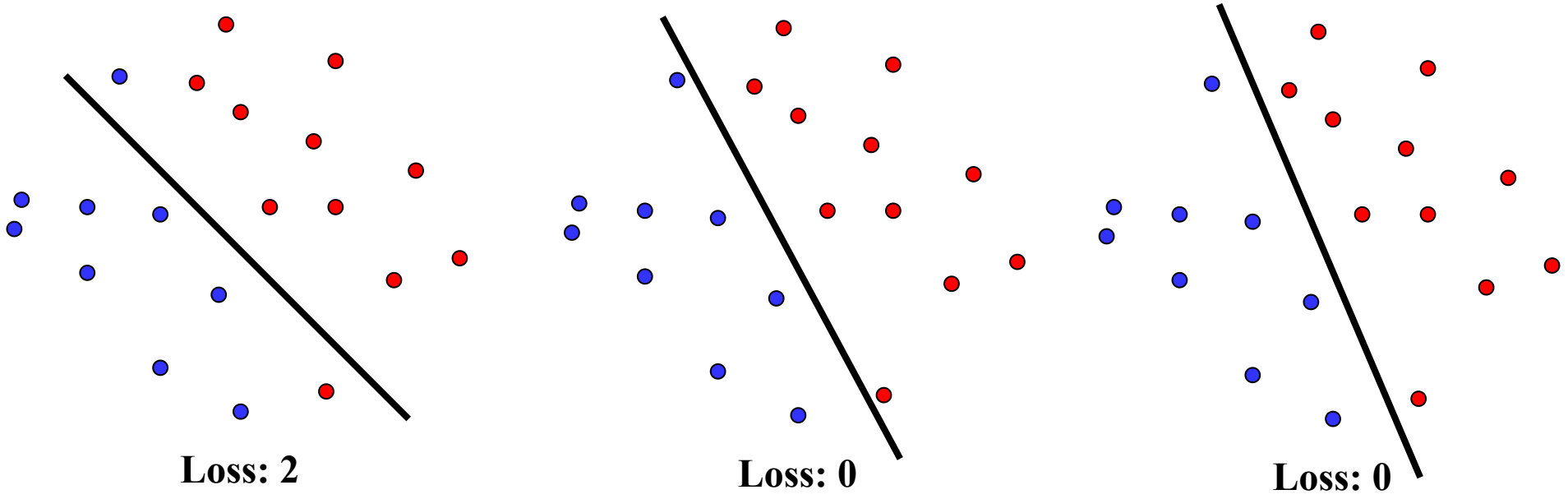


$$\mathbf{x}_i \text{ positive}: \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 0$$
$$\mathbf{x}_i \text{ negative}: \quad \mathbf{x}_i \cdot \mathbf{w} + b < 0$$

Which hyperplane is best? We need a **loss function** to decide

What is a good loss function?

- One possibility: Number of misclassified examples
  - Problems: discrete, can't break ties
  - We want the loss to lead to *good generalization*
  - We want the loss to work for more than 2 classes



Softmax classifier

- Interpret Scores as unnormalized log probabilities of classes

$$f(x_i, W) = Wx_i \quad \text{(score function)}$$

$$\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}$$ softmax function

Squashes values into *probabilities* ranging from 0 to 1

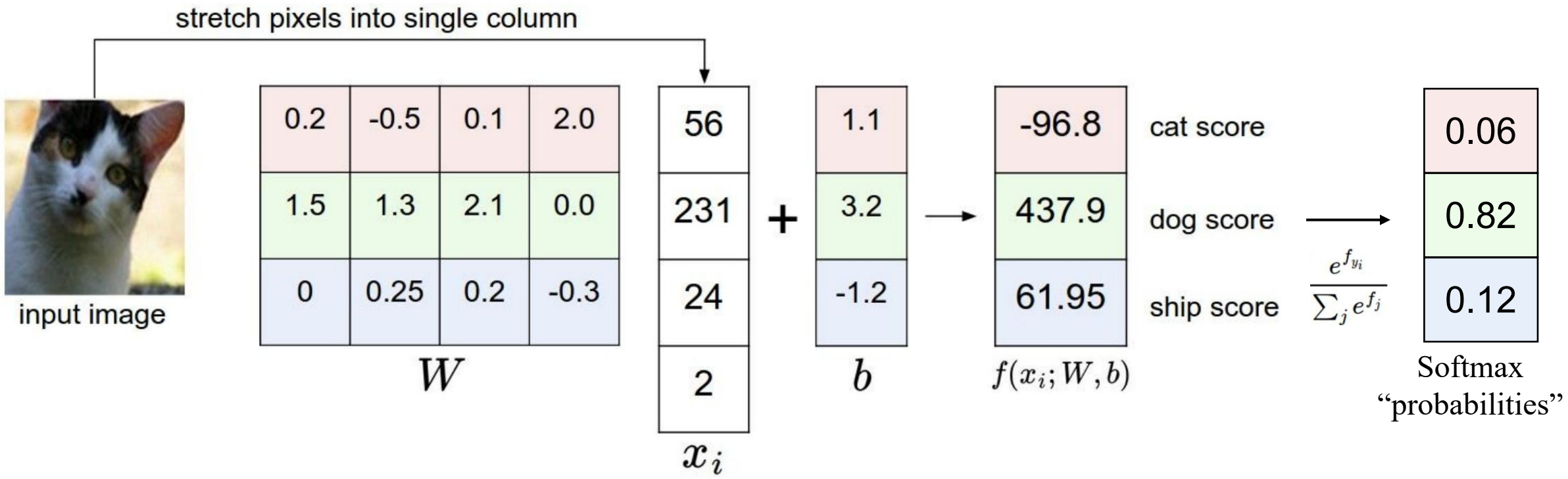
$$P(y_i \mid x_i; W)$$

Example with three classes:

$$[1, -2, 0] \rightarrow [e^1, e^{-2}, e^0] = [2.71, 0.14, 1] \rightarrow [0.7, 0.04, 0.26]$$

Softmax classifier

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



Cross-entropy loss

$$f(x_i, W) = Wx_i$$ (score function)

Cross-entropy loss

$$f(x_i, W) = Wx_i$$ (score function)

f_{y_i} : score of correct class

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)$$

$$L_i = -f_{y_i} + \log\sum_j e^{f_j}$$

We call L_i *cross-entropy loss*

Cross-entropy loss

$$f(x_i, W) = Wx_i$$ (score function)

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right) \qquad L_i = -f_{y_i} + \log\sum_j e^{f_j}$$

We call L_i *cross-entropy loss*

$P(y_i \mid x_i; W)$

i.e. we're minimizing the negative log likelihood.

Losses

- Cross-entropy loss is just one possible loss function
  - One nice property is that it reinterprets scores as probabilities, which have a natural meaning

- SVM (max-margin) loss functions also used to be popular
  - But currently, cross-entropy is the most common classification loss

The story so far: linear classification

- What we have: a score function and loss function
  - Score function maps an input data instance (e.g., an image) to a vector of scores, one for each category
  - Last time, our score function is based on linear classifier

$$f(x, W) = Wx + b$$

f: score function
x: input instance
W, b: parameters of a linear (actually affine) function

- Find **W** and **b** that minimize a *loss* over labeled training data, e.g. cross-entropy loss

$$L = \frac{1}{N} \sum_i -\log \left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}} \right)$$

Questions?

Neural networks

(**Before**) Linear score function: $f = Wx$

Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

(**Before**) Linear score function: $f = Wx$

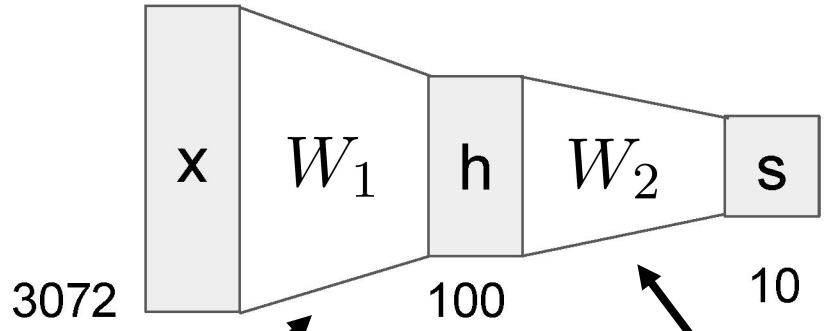
(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



W_1
(100 x 3072 matrix)

h
100D intermediate vector

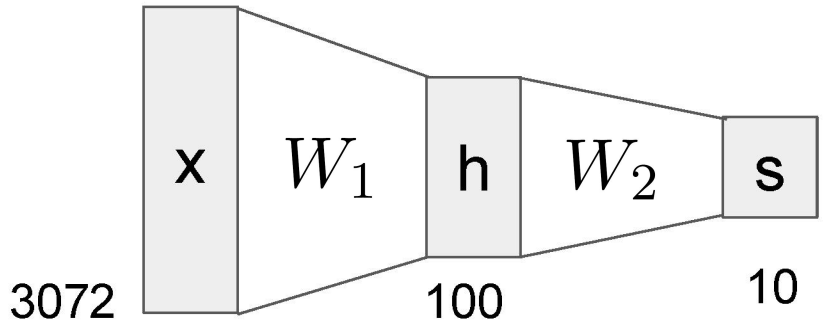
W_2
(10 x 100 matrix)

Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



- Total number of weights to learn:

  3,072 x 100 + 100 x 10 = 308,200

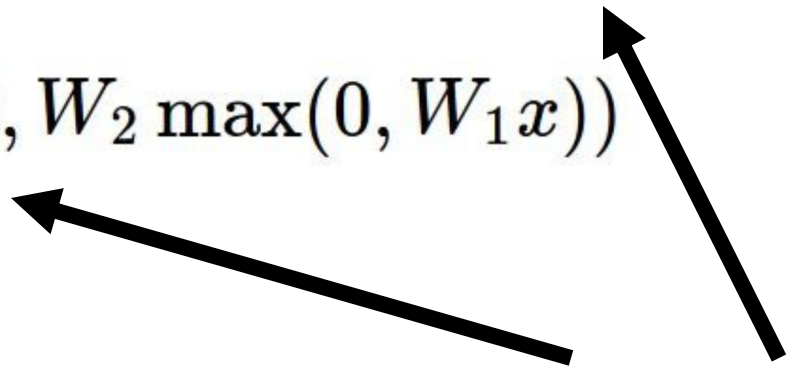
Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$



also called "**Multilayer Perceptrons**" (MLPs)

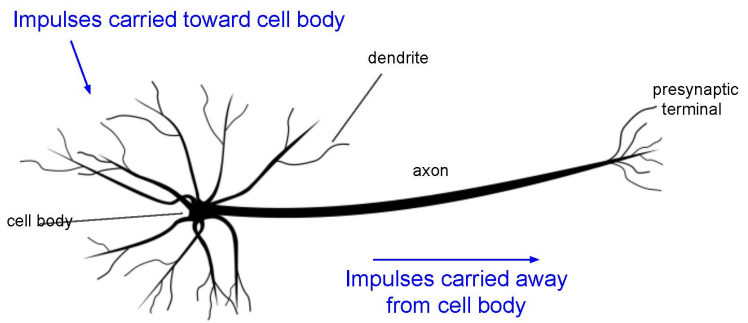
Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

- Very coarse generalization of neural networks:
  - Linear functions chained together and separated by non-linearities (*activation functions*), e.g. "max"

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

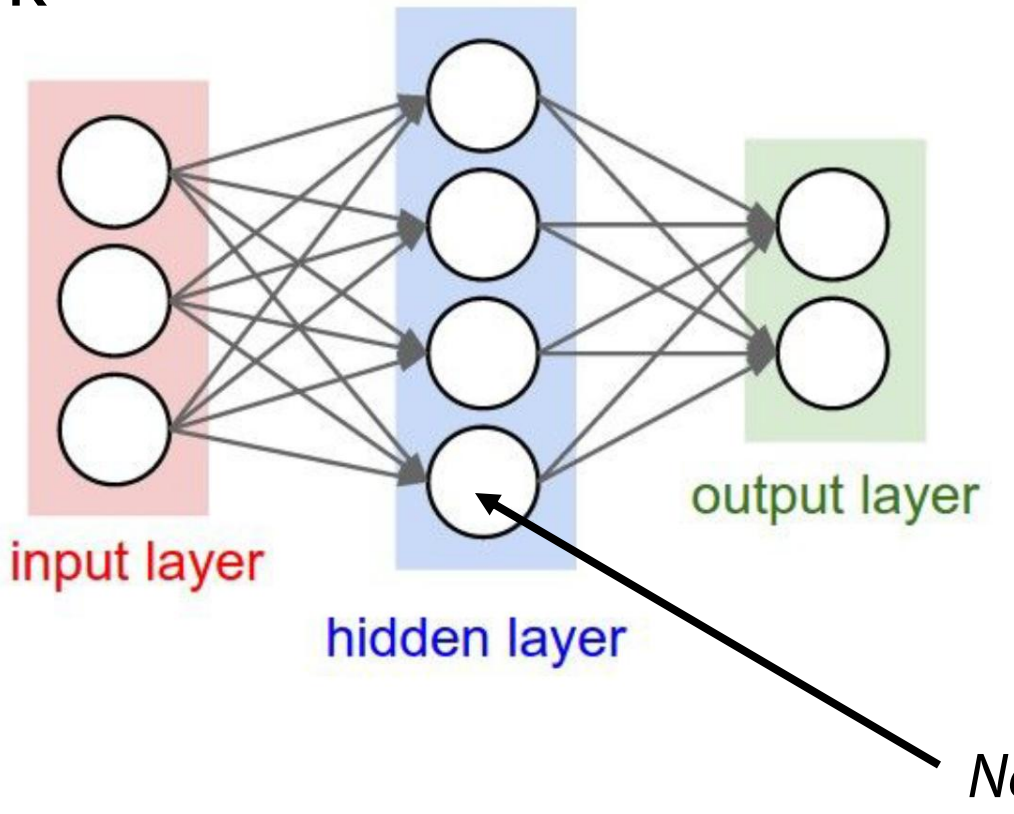
  - Why separate linear functions with non-linear functions?
  - *Very roughly* inspired by real neurons



This image by Felipe Perucho is licensed under CC-BY 3.0

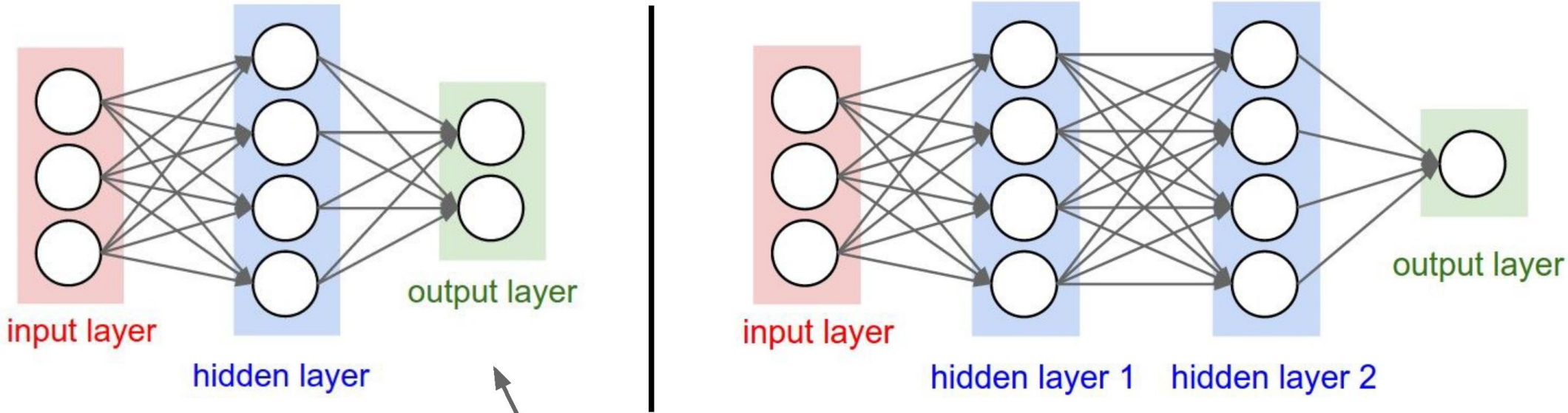
Neural network architecture

- Computation graph for a 2-layer neural network



Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks: architectures



"2-layer Neural Net", or
"1-hidden-layer Neural Net"

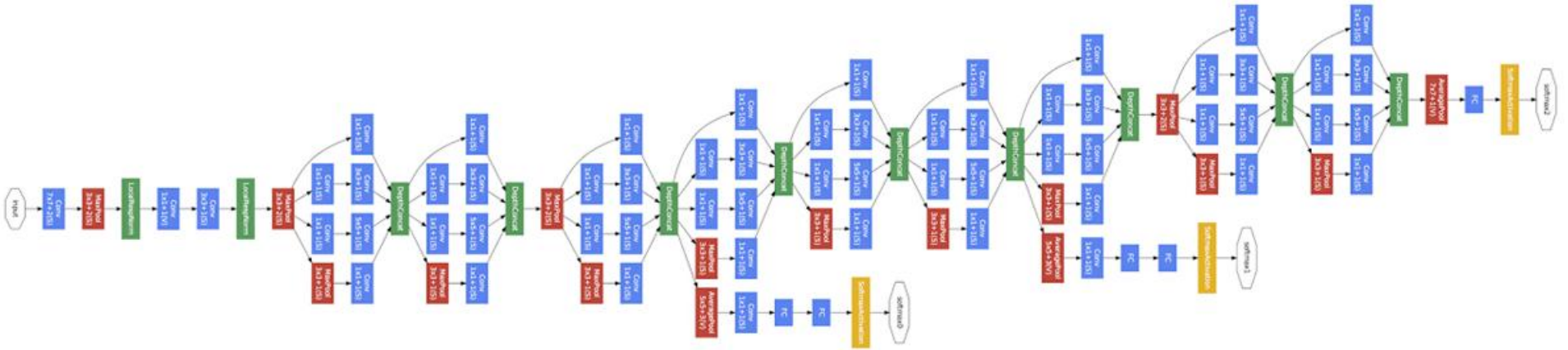
"3-layer Neural Net", or
"2-hidden-layer Neural Net"

"Fully-connected" layers

- **Deep** networks typically have many layers and potentially millions of parameters

Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Deep neural network

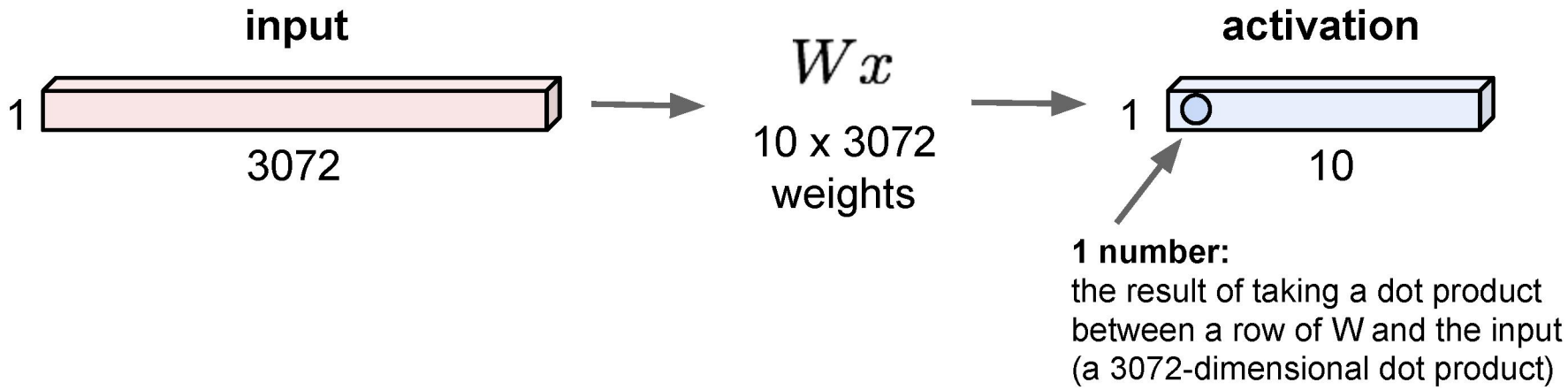


- *Inception* network (Szegedy et al, 2015)
- 22 layers

Adapted from Fei-Fei Li & Andrej Karpathy & Serena Leung

Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



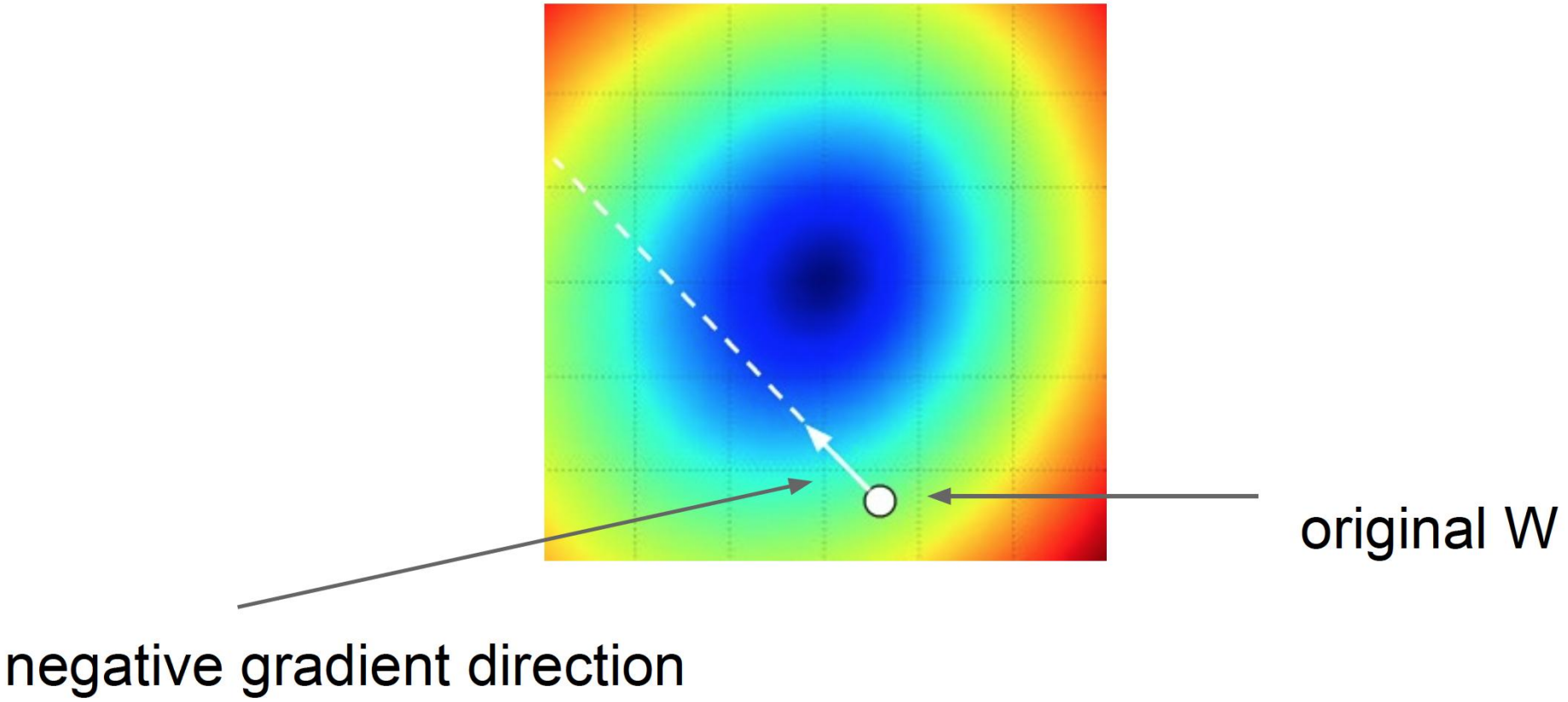
- Just like a linear classifer – but in this case, just one layer of a larger *network*

Summary so far

- A classic neural network arranges neurons into fully-connected layers. This kind of neural network is also called a **Multi-Layer Perceptron**
- The **layer** abstraction enables efficient implementations of neural networks using vectorized operations like matrix multiplication

Optimizing parameters with gradient descent

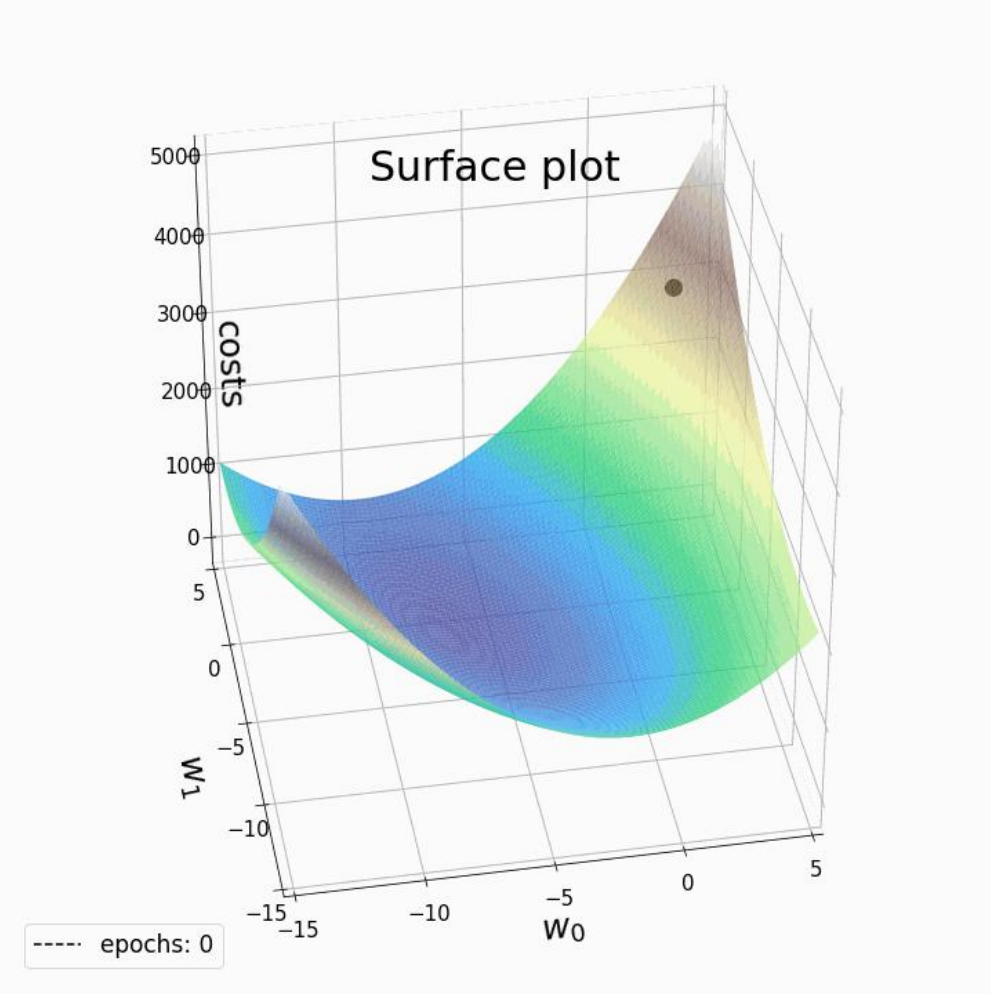
- How do we find the best **W** and **b** parameters?
- In general: *gradient descent*
  1. Start with a guess of a good **W** and **b** (or randomly initialize them)
  2. Compute the loss function for this initial guess and the *gradient* of the loss function
  3. Step some distance in the negative gradient direction (direction of steepest descent)
  4. Repeat steps 2 & 3
- Note: efficiently performing step 2 for deep networks is called *backpropagation*



Gradient descent: walk in the direction opposite gradient

- **Q**: How far?
- **A**: Step size: *learning rate*
- Too big: will miss the minimum
- Too small: slow convergence

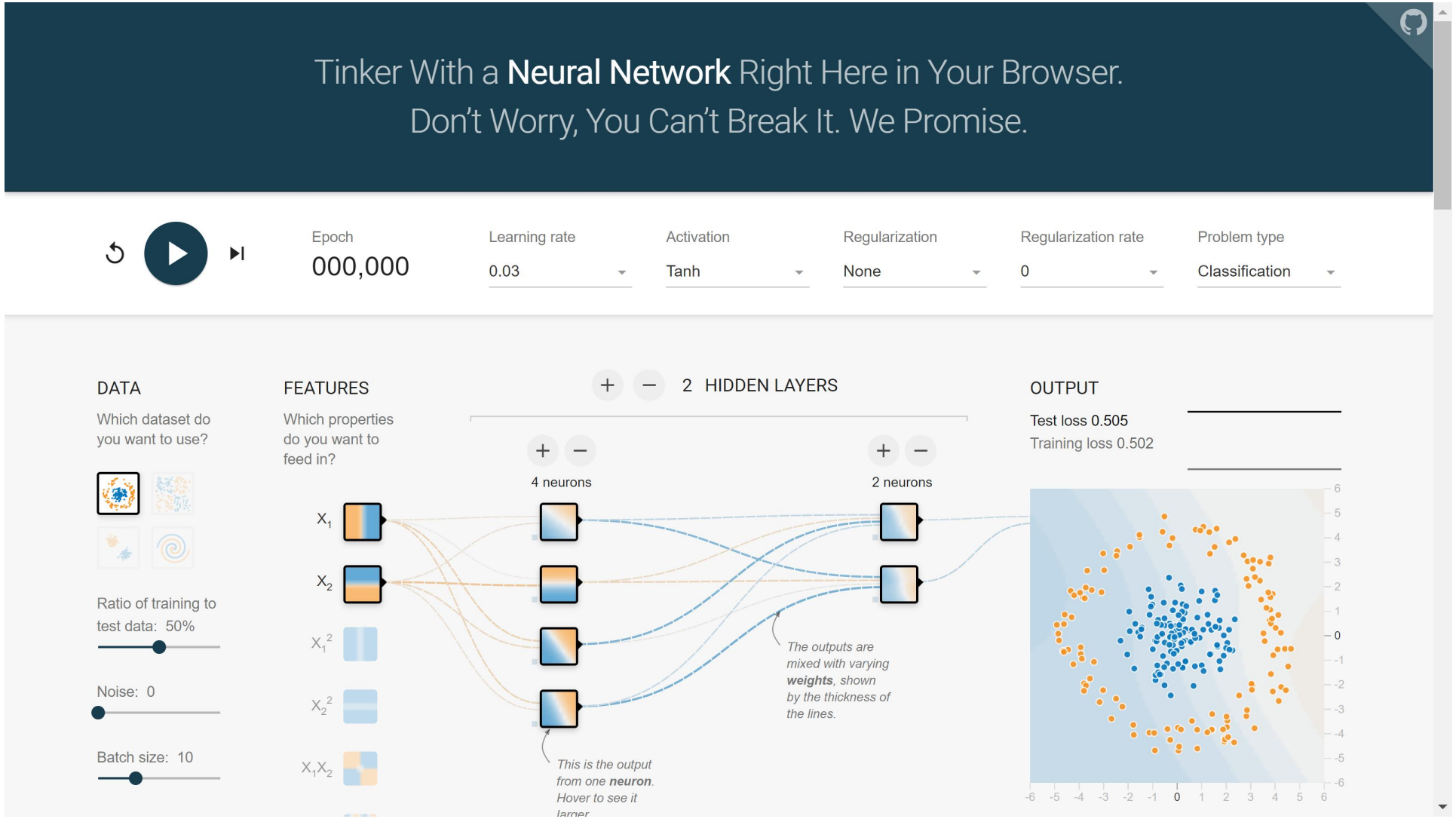
2D example of gradient descent



- In reality, in deep learning we are optimizing a highly complex loss function with millions of variables (or more)
- More on this later…

https://laptrinhx.com/gradient-descent-animation-2-multiple-linear-regression-3070246823/

2D example: TensorFlow Playground



https://playground.tensorflow.org

Questions?

Multi-layer perceptrons: just the beginning

- MLPs can work well with simple image classification problems (e.g., digit recognition)
- For more complex recognition problems (e.g., recognizing 1,000s of categories), we need more technology (convolutional neural networks and transformers)
- MLPs have a surprising application: 3D reconstruction!
  - See NeRFs (neural radiance fields) next time